→CSS PART-3

Position

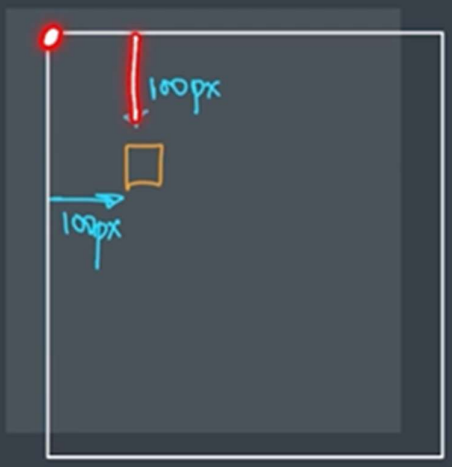
The position CSS property sets how an element is positioned in a document.

The top, right, bottom, and left properties determine the final location of positioned elements.

- static
- relative
- absolute
- fixed

Position - Static

The top, right, bottom, left, and z-index properties have no effect.
This is the default value.



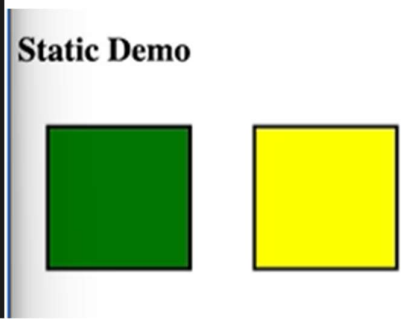
index.html > html > body > section

```
<meta http-equiv="X-UA-Compatible" content="IE=edge" />
<meta name="viewport" content="width=device-width, initial-scale=1.0"
<title>CSS</title>
<link rel="stylesheet" href="style.css" />
</head>
<body>
<section>
<h2>Static Demo</h2>
<div></div>
<div id="static"></div>
</section>
</body>
</html>
```

yle.css > #static

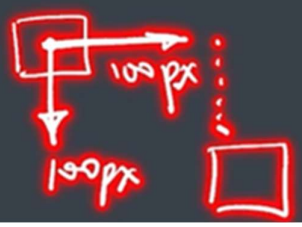
```
div {
  height: 100px;
  width: 100px;
  background-color: green;
  text-align: center;
  border: 3px solid black;
  margin: 20px;
  display: inline-block;
}

#static {
  background-color: yellow;
  position: static;
  top: 100px;
  left: 100px;
}
```



Position - Relative

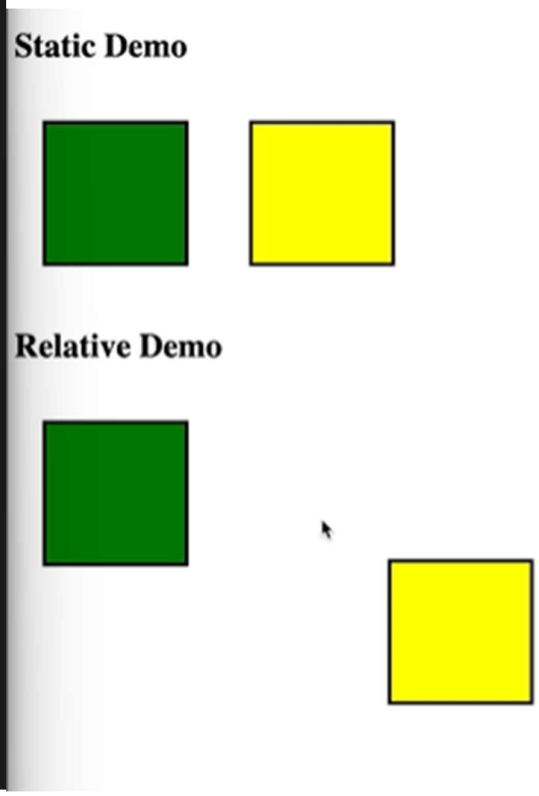
the offset is relative to itself based on the values of top, right, bottom, and left.



```
yle.css > #relative
div {
  height: 100px;
  width: 100px;
  background-color: green;
  text-align: center;
  border: 3px solid black;
  margin: 20px;
  display: inline-block;
}

#static {
  background-color: yellow;
  position: static;
  top: 100px;
  left: 100px;
}

#relative {
    background-color: yellow;
  position: static;
  top: 100px;
  left: 100px;
}
```

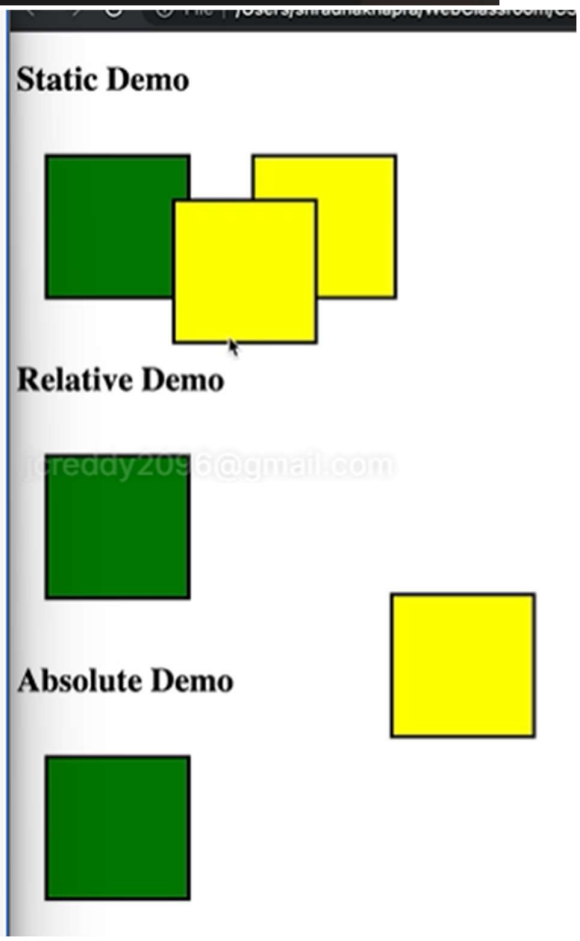


Position - Absolute

The element is removed from the normal document flow, and no space is created for the element in the page layout.
It is positioned relative to its closest positioned ancestor, if any; otherwise, it is placed relative to the initial containing block.

```
<meta charset="UTF-8" />
<meta http-equiv="X-UA-Compatible" content="IE=edge" />
<meta name="viewport" content="width=device-width, initial-scale=1.0" />
<title>CSS</title>
<link rel="stylesheet" href="style.css" />
</head>
<body>
<section>
<h2>Static Demo</h2>
<div></div>
<div id="static"></div>
</section>
<section>
<h2>Relative Demo</h2>
<div></div>
<div id="relative"></div>
</section>
<section>
<h2>Absolute Demo</h2>
<div></div>
<div id="absolute"></div>
</section>
</body>
</html>
```

abc Absolute

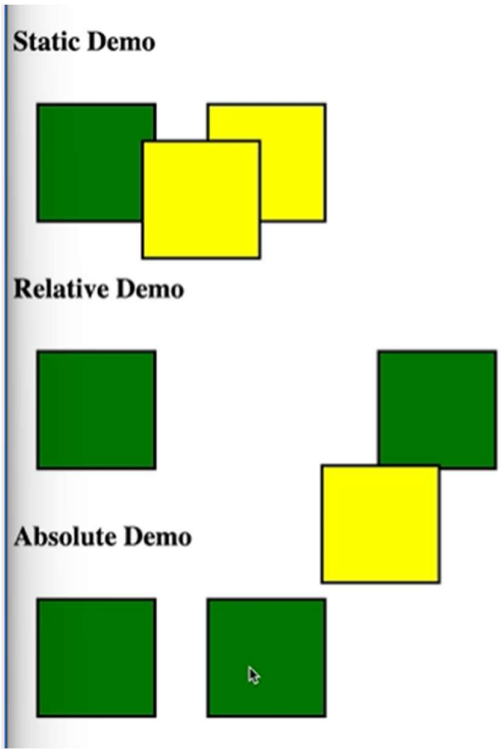



```
#absolute {
    background-color: yellow;
  position: absolute;
  top: 100px;
  left: 100px;
}
```

absolute

The box's posi
'right', 'bottom

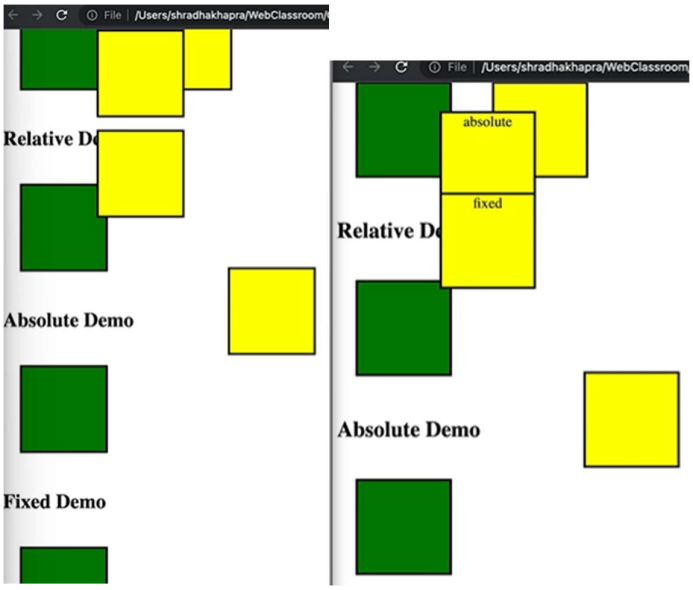
→if three divs are created




```
  <meta http-equiv="X-UA-Compatible" content
  <meta name="viewport" content="width=devic
  <title>CSS</title>
  <link rel="stylesheet" href="style.css" />
</head>
<body>
  <section>
    <h2>Static Demo</h2>
    <div></div>
    <div id="static"></div>
  </section>
  <section>
    <h2>Relative Demo</h2>
    <div></div>
    <div id="relative"></div>
    <div></div>
  </section>
  <section>
    <h2>Absolute Demo</h2>
    <div></div>
    <div id="absolute"></div>
    <div></div>
  </section>
</body>
```

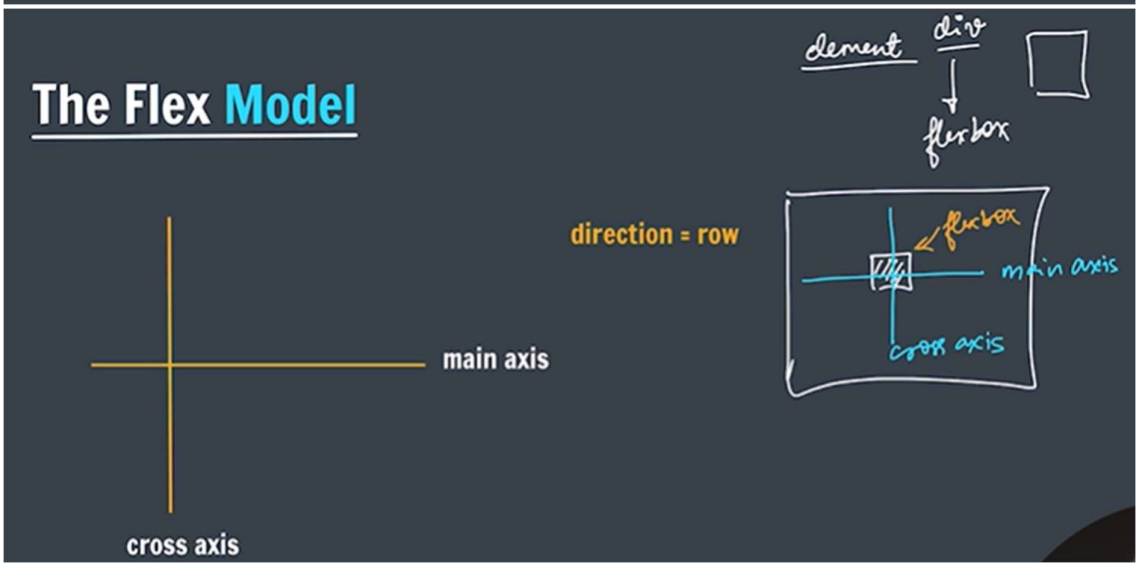
Position - Fixed

The element is removed from the normal document flow, and no space is created for the element in the page layout.
It is positioned relative to the initial containing block established by the viewport.

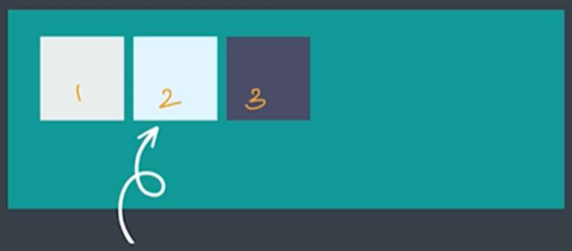


→Creating a smileyface

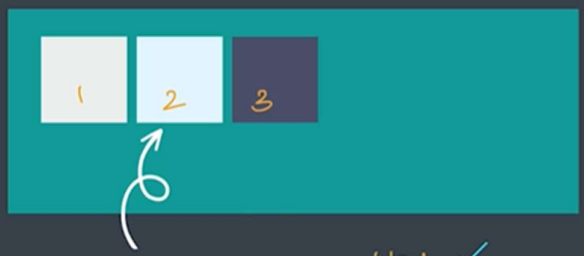
→what is flexbox



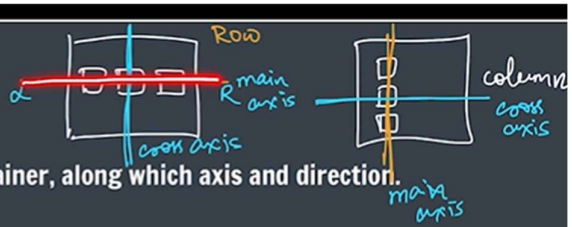
General Example



General Example



Flexbox Direction



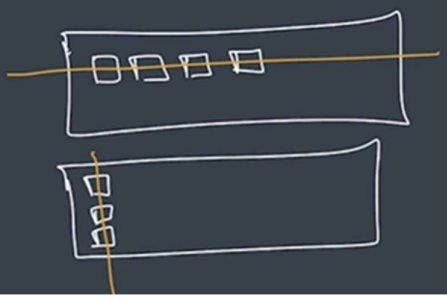
It sets how flex items are placed in the flex container, along which axis and direction.

- flexbox-direction : row; main axis, left to right
- flexbox-direction : row-reverse; main axis, right to left
- flexbox-direction : column; main axis, top to bottom
- flexbox-direction : column-reverse; main axis, bottom to top

Justify Content

Tells how the browser distributes space between and around content items along the main-axis

- justify-content : flex-start;
- justify-content : flex-end;
- justify-content : center;



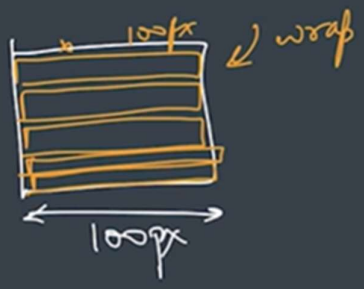
Justify Content

- justify-content : space-between;
- justify-content : space-around;
- justify-content : space-evenly;

Flex Wrap

Sets whether flex items are forced onto one line or can wrap onto multiple lines

- flex-wrap : nowrap;
- flex-wrap : wrap;
- flex-wrap : wrap-reverse;



cross axis
reverse direction
wrap

Align Items

Distributes our items along the cross axis

- justify-content : flex-start;
- justify-content : flex-end;
- justify-content : center;

Align Items

- justify-content : baseline;

Align Content

it sets the distribution of space between and around content items along a flexbox's cross-axis

- align-content : flex-start / flex-end / center
- align-content : space-between / space-around / evenly
- align-content : baseline;

Align Self

Aligns an item along the cross axis

- align-self : flex-start;
- align-self : flex-end;
- align-self : center;
- align-self : baseline;

Flex Sizing

- flex-basis

It sets the initial main size of a flex item.

flex-basis : 100px;

Flex Sizing

- flex-grow

It specifies how much of the flex container's remaining space should be assigned to the flex item's main size

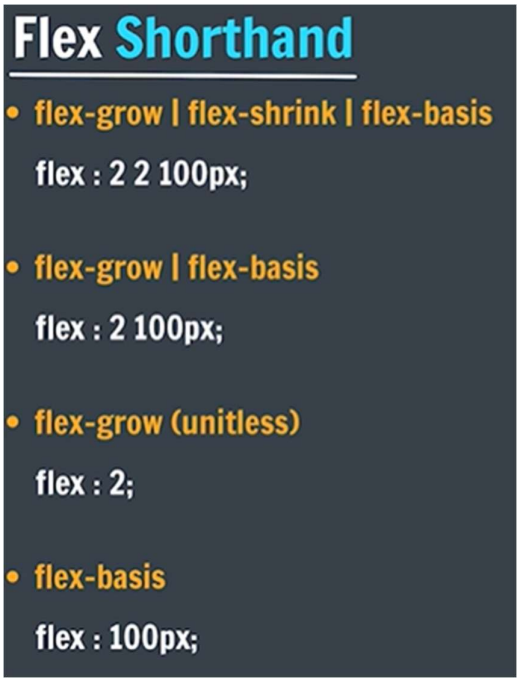
flex-grow : 1 (default)

Flex Sizing

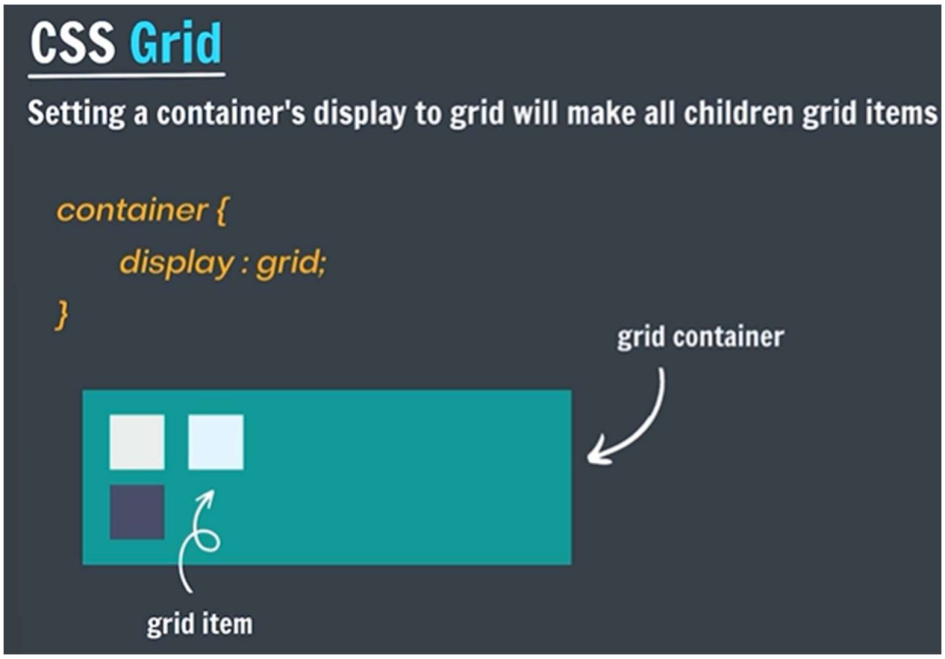
- flex-shrink

It sets the flex shrink factor of a flex item.

flex-shrink : 1


Flex Shorthand
• flex-grow | flex-shrink | flex-basis
flex : 2 2 100px;
• flex-grow | flex-basis
flex : 2 100px;
• flex-grow (unitless)
flex : 2;
• flex-basis
flex : 100px;

→CSS Part-6

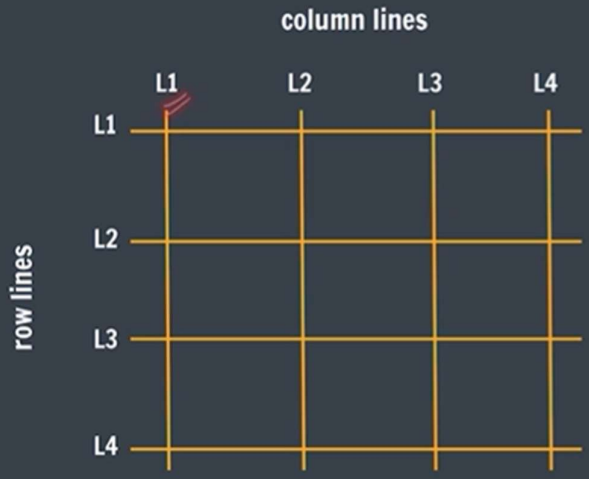

CSS Grid
Setting a container's display to grid will make all children grid items
container {
display : grid;
}
grid container
grid item

Grid Model

Grid Lines

Grid Cell

Grid Track



Grid Template

They define the lines & track sizing

grid-template-rows

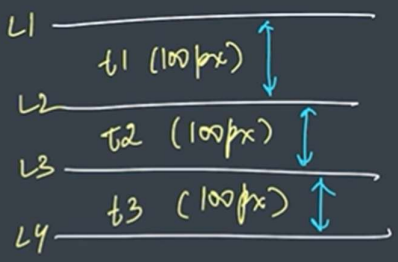
grid-template-columns

Grid Template

They define the lines & track sizing

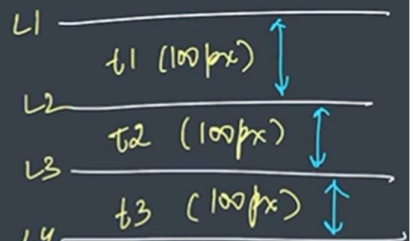
grid-template-rows :

grid-template-columns



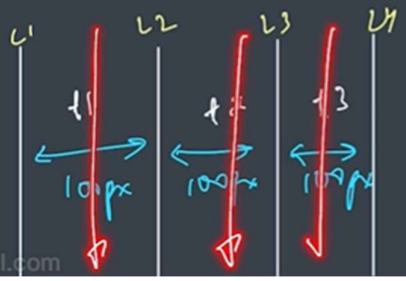
Grid Template

They define the lines & track sizing



grid-template-rows : 100px 100px 100px;

grid-template-columns



Grid Template

Repeat is used to divide all available space

grid-template-rows : repeat(count, 1fr)

grid-template-columns : repeat(count, 1fr)

grid-template-rows : repeat(2, 1fr)

grid-template-rows : 1fr 1fr

Grid Gaps

They define the gaps between the lines

row-gap : 10px;

column-gap : 10px;

grid-gap : rowGap columnGap

Grid Columns

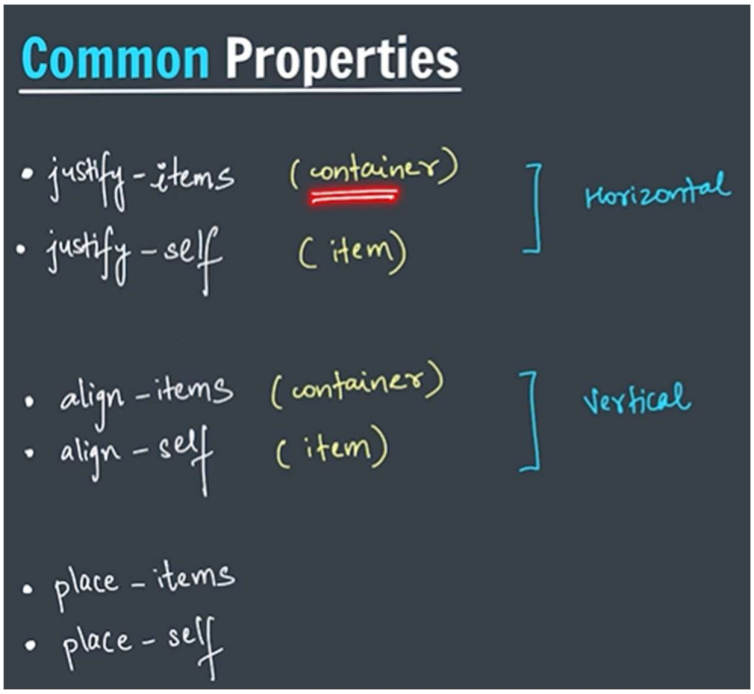
Defines an item's starting & ending position inside the column

grid-column-start : line_number

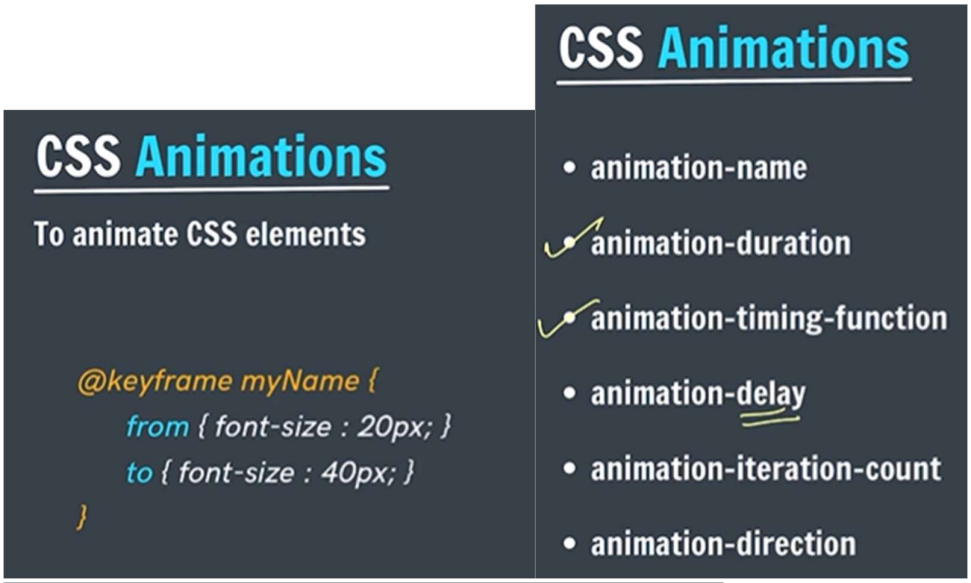
grid-column-end : line_number

grid-column : start_col / end_col

grid-column : start_col / span number


Common Properties
justify-items (container)
justify-self (item)
Horizontal
align-items (container)
align-self (item)
Vertical
place-items
place-self

→inline grid


CSS Animations
To animate CSS elements
@keyframe myName {
from { font-size : 20px; }
to { font-size : 40px; }
}
CSS Animations
animation-name
animation-duration
animation-timing-function
animation-delay
animation-iteration-count
animation-direction


Animation Shorthand
animation : myName 2s linear 3s infinite normal

% in Animation


```
@keyframe myName {
    0% { font-size : 20px; }
    50% { font-size : 30px; }
    100% { font-size : 40px; }
}
```



```
@keyframes colAnimate {
  0% {
    background-color: green;
  }
  50% {
    background-color: yellow;
  }
  80% {
    background-color: red;
  }
  100% {
    background-color: blue;
  }
}
```

Media Queries

width (of viewport)

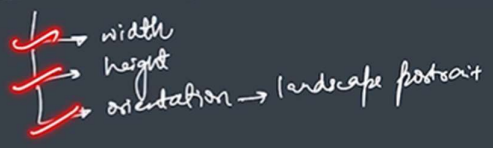
```
@media (min-width : 400px) {
    div {
        background-color : red;
    }
}
```

```
@media (max-width : 400px) {
    div {
        background-color : red;
    }
}
```

Media Queries

media features - width (of viewport)

device / environment




```
@media (width : 400px) {
    div {
        background-color : red;
    }
}
```

Media Queries

width (of viewport)

```
@media (min-width : 200px) and (min-width : 300px) {
    div {
        background-color : red;
    }
}
```

(min → max)

Media Queries

orientation (of viewport)

```
@media (orientation : landscape) {
    div {
        background-color : red;
    }
}
```

z-index

position - static, default
X

It decides the stack level of elements

Overlapping elements with a larger z-index cover those with a smaller one.

z-index : auto (0)

(+ve) *z-index : 1 / 2 / ...* upper

(-ve) *z-index : -1 / -2 / ...* lower